



δ LTA: Decoupling Camera Sampling from Processing to Avoid Redundant Computations in the Vision Pipeline

Raúl Taranco
taranco@ac.upc.edu
Universitat Politècnica de Catalunya
Barcelona, Spain

José-María Arnau
jarnau@ac.upc.edu
Universitat Politècnica de Catalunya
Barcelona, Spain

Antonio González
antonio@ac.upc.edu
Universitat Politècnica de Catalunya
Barcelona, Spain

ABSTRACT

Continuous Vision (CV) systems are essential for emerging applications like Autonomous Driving (AD) and Augmented/Virtual Reality (AR/VR). A standard CV System-on-a-Chip (SoC) pipeline includes a frontend for image capture and a backend for executing vision algorithms. The frontend typically captures successive similar images with gradual positional and orientational variations. As a result, many regions between consecutive frames yield nearly identical results when processed in the backend. Despite this, current systems process every image region at the camera's sampling rate, overlooking the fact that the actual rate of change in these regions could be significantly lower.

In this work, we introduce δ LTA (δ ont't Look Twice, it's Alright), a novel frontend that decouples camera frame sampling from backend processing by extending the camera with the ability to discard redundant image regions before they enter subsequent CV pipeline stages. δ LTA informs the backend about the image regions that have notably changed, allowing it to focus solely on processing these distinctive areas and reusing previous results to approximate the outcome for similar ones. As a result, the backend processes each image region using different processing rates based on its temporal variation.

δ LTA features a new Image Signal Processing (ISP) design providing similarity filtering functionality, seamlessly integrated with other ISP stages to incur zero-latency overhead in the worst-case scenario. It also offers an interface for frontend-backend collaboration to fine-tune similarity filtering based on the application requirements.

To illustrate the benefits of this novel approach, we apply it to a state-of-the-art CV localization application, typically employed in AD and AR/VR. We show that δ LTA removes a significant fraction of unneeded frontend and backend memory accesses and redundant backend computations, which reduces the application latency by 15.22% and its energy consumption by 17%.

CCS CONCEPTS

• **Computing methodologies** $\rightarrow$ **Computer vision**; • **Computer systems organization** $\rightarrow$ **System on a chip**.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MICRO '23, October 28–November 01, 2023, Toronto, ON, Canada

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0329-4/23/10...\$15.00

<https://doi.org/10.1145/3613424.3614261>

KEYWORDS

Image Signal Processor, Image Similarity, Computation Reuse

ACM Reference Format:

Raúl Taranco, José-María Arnau, and Antonio González. 2023. δ LTA: Decoupling Camera Sampling from Processing to Avoid Redundant Computations in the Vision Pipeline. In *56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '23)*, October 28–November 01, 2023, Toronto, ON, Canada. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3613424.3614261>

1 INTRODUCTION

Video cameras are capable of capturing images with ever-increasingly high resolution and frame rates [28, 42, 63, 86]. At the same time, emerging applications using these cameras, such as Autonomous Driving [90] (AD), Augmented and Virtual Reality [14] (AR/VR), smart vision [2], and robotics, are rapidly evolving. Their sophisticated algorithms necessitate substantial computational resources with tightly energy-constrained budgets. Additionally, most of these applications are deployed in mobile systems, requiring low-power solutions. Consequently, these Continuous Vision (CV) systems face significant challenges in delivering accurate results in a timely and energy-efficient manner.

The major challenge for CV systems is latency, which is especially critical in worst-case scenarios. For instance, CV systems must react quickly to environmental changes in AD vehicles to ensure safe and efficient navigation. With cars capable of traversing 2-3 meters in just 100 ms, even the slightest delay can have dramatic consequences [20]. Another difficulty is ensuring a good user experience, notably in applications such as AR, where users may experience dizziness and discomfort if the system is not responsive enough. Energy consumption also imposes significant limitations and exacerbates previous challenges, particularly in battery-powered systems where autonomy is critical.

A standard CV System-on-a-Chip (SoC) pipeline includes a frontend for image capture and a backend for executing vision algorithms. The frontend captures the image frames and writes them in a Framebuffer (FB) in the main memory of the SoC. The backend, in turn, reads the images from the FB and processes them. In CV, the camera movement is frequently smooth, meaning that the camera captures images of the same scene with gradual changes in position, angle, and orientation. This smoothness results in consecutive frames having a high degree of temporal similarity. For example, when an autonomous vehicle stops at a red traffic light, it processes similar frames. Likewise, when the vehicle begins to move again, some image regions, such as the sky or road, change minimally.

In this work, we leverage this smoothness property of input vision streams to propose δ LTA (δ ont't Look Twice, it's Alright).

δ LTA optimizes CV by decoupling frontend camera frame sampling from the backend processing. We introduce a new CV frontend that utilizes frame-to-frame similarity to discard image regions in the very early stages of the vision pipeline during the frontend camera sampling. Instead of writing the entire frame to the FB every time the camera captures an image, δ LTA frontend selectively updates only the regions of the frame that differ from the previous version and informs the backend which regions are different. This mechanism allows the backend to focus solely on processing the unique and distinctive regions of the image at their rate of change and not always at the arbitrary camera sampling rate.

Compared with current systems, δ LTA reduces the frontend FB updates, the redundant backend computations, and the backend FB accesses needed to produce equivalent final results with decreased end-to-end latency and overall energy consumption. Another pivotal benefit of decoupling the backend workload from the sampling arises when considering camera frame rate scaling. Frame-to-frame similarity increases as the frame rate grows since the camera has less time to capture variations in the scene between consecutive frames up to a certain point when, for a given scene, more frame rate provides only redundant information. Hence, δ LTA can progressively filter the increasingly similar regions of these frames to limit the backend workload to the actual scene information. Moreover, increasing the frame rate enhances responsiveness. To illustrate it, consider a baseline system limited to processing streams at 20 FPS. Consider also that, assuming the particular temporal similarity characteristic of a given scene, this system with the δ LTA frontend can process instead, 30 FPS. While the baseline system processes all image regions at 20 FPS, δ LTA-equipped one can selectively process some regions, the most distinctive that changed, at up to 30 FPS, thanks to freeing computational resources that otherwise would be devoted to processing irrelevant regions.

To ensure all these advantages, δ LTA must efficiently address two notable challenges. The first challenge is enabling the frontend to detect similar regions efficiently without significant latency and energy consumption overheads. To achieve this, δ LTA divides each frame into a grid of equally sized regions and computes a BRIEF [11] signature for each region through a specialized hardware accelerator embedded in the Image Signal Processor (ISP) of the frontend. BRIEF signatures require a limited number of pixel computations. They can be efficiently stored in a small buffer in the ISP, as they only occupy a fraction of the original region. To identify similar regions in an incoming frame, the ISP computes the Hamming distance between the BRIEF signatures for each region of the incoming frame and the past frame stored in a buffer. We show that this approach is highly efficient and requires a relatively small accelerator.

The second challenge of δ LTA is to design a versatile frontend interface since CV systems operate in a wide variety of potential scenarios. We achieve this flexibility through a hardware-software co-design and collaboration approach. The proposed frontend interface offers the programmer two primary mechanisms to regulate its adaptable behavior: i) a threshold that determines when δ LTA ISP should consider that two regions are similar and ii) a mask that specifies which regions it must update regardless of the similarity measurement. The vision processing software can be modified to take advantage of the new frontend interface using the

threshold and mask according to the particular characteristics and requirements of the CV system, its deployment, and the acceptable approximation levels.

δ LTA has broad applicability to vision systems. We illustrate its effectiveness in AD, representing a crucial CV application that is expected to revolutionize transportation. Specifically, we focus on the CV pipeline used by state-of-the-art AD systems [30, 51, 52, 54, 89], which rely on mobile, energy-efficient SoCs to perform real-time camera-based localization achieved through Simultaneous Localization and Mapping [10, 16, 27, 32, 45, 77, 88] (SLAM) algorithms. AD systems are subject to strict latency and energy constraints, making them an ideal test bed for our approach.

We show that δ LTA provides significant benefits at the expense of small extensions to current mobile CV SoCs. In particular, δ LTA achieves up to a 15.22% reduction in latency and a 17% reduction in energy consumption.

In summary, the key contributions of this work are:

- We propose δ LTA, a novel, decoupled visual frontend for contemporary CV SoCs. δ LTA exposes which regions of a frame have changed with respect to the previous one, enabling many opportunities for computation reuse and approximate computing.
- We propose an ISP architecture that supports the new frontend based on a highly efficient method to discriminate similarity in image regions based on BRIEF [11].
- We show how to co-design the software with δ LTA to achieve fast, low-power continuous vision localization.
- We compare the performance of our solution with the baseline and with an optimized backend implementation, showing that the new interface significantly reduces latency and energy consumption. δ LTA also enables CV SoCs to process higher frame rates and enhance responsiveness.

We organize the remainder of this document as follows: Section 2 provides background used later in the paper. Section 3 describes the hardware architecture of δ LTA and how software can use it. Section 4 details the evaluation methodology. Section 5 presents the performance, energy, and accuracy results. Section 6 reviews related work, and finally, Section 7 summarizes the main conclusions.

2 BACKGROUND AND MOTIVATION

This section provides an overview of our research context. We first discuss the state-of-the-art Continuous Vision (CV) pipeline and its applications in camera-based localization, widely used in emerging technologies like Autonomous Driving (AD) and Augmented/Virtual Reality (AR/VR) [45]. We then examine the spatio-temporal redundancy inherent in CV inputs and the BRIEF method that we leverage to detect it. Finally, we discuss how previous studies have exploited spatio-temporal redundancy through software caching.

2.1 Continuous Vision Pipeline

CV systems have two primary components: a frontend and a backend (Figure 1a). The frontend processes pixel data, while the backend extracts high-level semantic information for decision-making. Off-chip camera sensors capture light, generating RAW pixels sent to the SoC via the MIPI Camera Serial Interface (CSI) [50]. Within

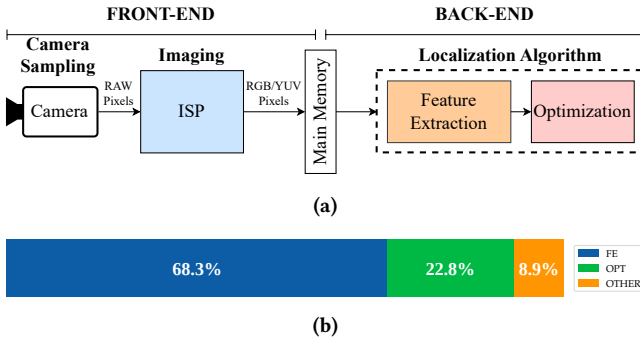


Figure 1: (1a) Depiction of a CV pipeline, highlighting the front and backend that embeds a visual localization system. (1b) Relative execution time for our localization system.

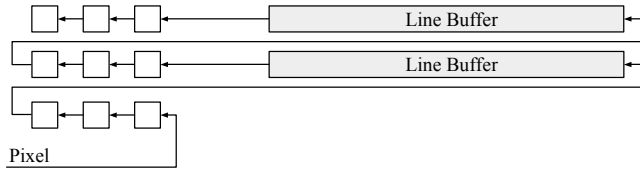


Figure 2: Scheme of a typical 2D line buffer employed on image processing specialized hardware.

the SoC, the Image Signal Processor (ISP) is a specialized SoC component that transforms Bayer domain RAW data into RGB/YUV domain pixels using algorithms like dead pixel correction, demosaicing, and white-balancing [66]. The ISP design involves a cascaded stencil operations pipeline on local SRAMs organized as line buffers [9, 38] (shown in Figure 2). Stencil operations employ this structures [81] to process an input pixel window and generate output pixels, typically following the image storage pattern using raster scan order.

2.2 Camera-based Localization Overview

Localization is an essential task that allows a moving agent (e.g., a car) to determine its position and orientation in a given frame of reference. Contemporary camera-based AD localization systems are composed of two main stages: Feature Extraction (FE) stage and Optimization (OPT) stage [30] (see Figure 1a).

The FE stage plays a crucial role in these systems. It performs several image processing methods and extracts visual features from every incoming camera frame to find visual correspondences in consecutive observations. This process is sensitive to the camera's resolution and frame rate. According to our experiments (see Figure 1b), this stage spends up to 60-70% of the total execution time, clearly establishing it as the main bottleneck of the application.

On the other hand, the OPT stage calculates the position (x, y, z coordinates) and orientation (yaw, roll, and pitch) of the agent using the visual feature correspondences generated in the FE stage.

In this work, we experiment with a state-of-the-art [45] approach called ORB-SLAM [12, 61] in localization-only mode to illustrate the benefits and applicability of our idea. However, it is worth noting

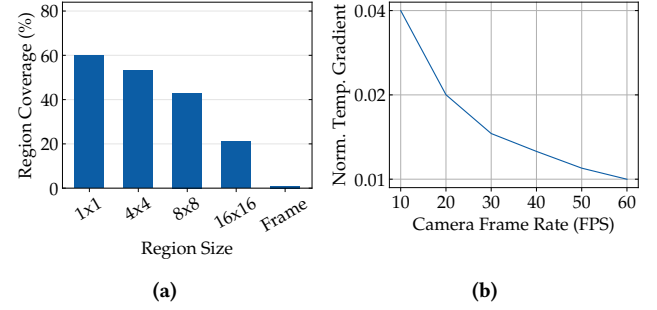


Figure 3: (3a) Similarity study at different region sizes across adjacent frames for sequence 00 of the KITTI [33] dataset. (3b) Reduction in Normalized Temporal Gradient with increasing frame rate (FPS).

that other recent approaches, such as SOFT2 [19] (currently the top-ranked in the KITTI [33] benchmark), could also benefit from our contribution.

2.3 Spatio-temporal Similarity

In CV, camera motion is often smooth, resulting in images that display subtle shifts in position, angle, and direction. This smoothness causes adjacent frames to exhibit significant spatio-temporal similarity. In this work, we leverage this spatio-temporal similarity to optimize the processing of adjacent frames. Figure 3a presents a similarity study that divides each image into a grid of varying-size regions to assess the percentage of identical regions between consecutive frames. As region sizes increase, the percentage of identical regions decreases significantly. For instance, around 60% of 1x1 regions (i.e., pixels) are identical, whereas only 20% of 16x16 regions share this characteristic. For larger regions, the percentage drops considerably, reaching nearly 0% of whole identical frames.

However, opportunities for computation reuse are much higher if we consider similar regions rather than totally identical ones. Figure 4 provides an example using 64x64 pixel regions, demonstrating that many regions exhibit minimal changes during typical camera movements. Several factors contribute to this observation, such as the parallax effect, camera sampling-to-velocity ratio, and low 2D frequency component areas.

The parallax effect causes objects far from the camera to appear to move slowly relative to the camera. Figure 4 exemplify such areas in blue.

The camera sampling-to-velocity ratio also impacts the frame-to-frame similarity. Figure 3b clearly illustrates the relationship between an increase in frame rate (FPS) and similarity within a given scene. We assess similarity using a metric called Temporal Gradient (TG), a widely accepted indicator for temporal similarity in video processing [84]. It involves computing the overall difference between the pixel values of two consecutive frames by subtracting one frame from the other and then averaging the resulting pixel values across the entire image. A low TG indicates little change between consecutive frames, implying high similarity. The normalized TG (by the maximum brightness) consistently decreases as

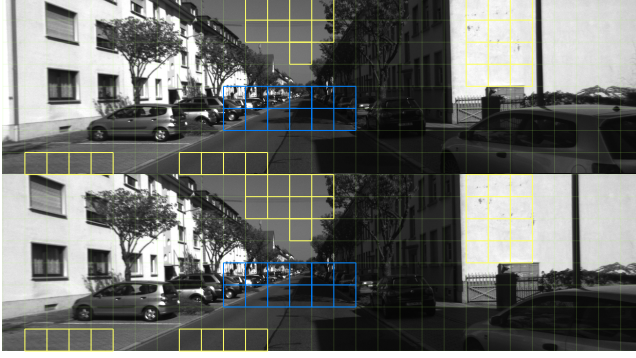


Figure 4: The figure compares two consecutive KITTI frames and highlights regions that exhibit spatio-temporal similarity. Yellow regions represent areas with low 2D frequency and high similarity. The blue regions correspond to distant areas in the scene that move slowly due to parallax.

the frame rate increases, indicating that higher frame rates lead to higher similarity.

Another property of a CV stream is related to the low 2D frequency areas. These areas, containing less information, require lower frame rates than more detailed regions. Figure 4 highlights similar regions in yellow. Despite movement between frames, many of these regions maintain nearly identical values due to the uniformity of their surrounding area.

2.4 BRIEF Overview

BRIEF, or Binary Robust Independent Elementary Features, is a method to generate a concise representation or signature of an image region's pixel intensity distribution [11]. BRIEF describes regions as the concatenation of the results of intensity comparisons between sampled pixel pairs within the region.

The generation of the coordinates of the pixel pairs, n in total, is a fundamental aspect of BRIEF. This pattern is established once and consistently applied when constructing BRIEF descriptors across all the image regions intended for similarity comparison. Typically, pixel pair positions can be chosen either through a random sampling within the image region or through a more sophisticated learning-based approach where machine learning aids in selecting the optimal pairs based on training data [11]. Our proposal uses the OpenCV [6] BRIEF implementation pattern of pair coordinates.

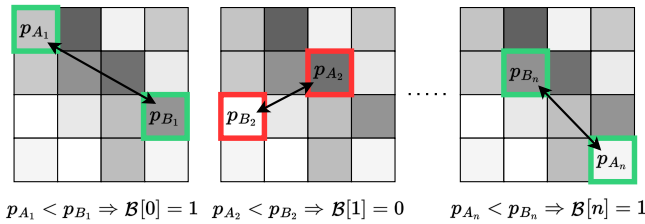


Figure 5: Example of the BRIEF ($\mathcal{B}$) operation to form the signature 10...1.

Given an image intensity function I , a binary test τ on an image patch with pixel coordinates p_A and p_B is defined as:

$$\tau(p_A, p_B) = \begin{cases} 1 & \text{if } I(p_A) < I(p_B) \\ 0 & \text{otherwise} \end{cases}$$

The BRIEF descriptor $\mathcal{B}$ using n binary tests is:

$$\mathcal{B} = [\tau(p_{A_1}, p_{B_1}), \dots, \tau(p_{A_n}, p_{B_n})]$$

Figure 5 depicts a representation of the BRIEF method operation. For a given pixel pair, p_A , and p_B , if the intensity of p_A is greater or equal to p_B , the bit is set; otherwise, it is unset. We use the Hamming distance of the BRIEF signatures of two regions to evaluate their similarity with a threshold-based classifier.

2.5 Software Caching

A variety of prior works exploit the spatio-temporal similarity in CV. Some employ software-based solutions as a cache service to minimize computational redundancy across simultaneous vision applications. Starfish [49] identifies library calls with identical arguments, enabling the reuse of computation results. CBinfer [13] and DeepMon [40] employ CNN caches, allowing computation reuse for each layer by matching image regions within a uniformly sized grid. Potluck [36] facilitates cross-application deduplication of image region computations through a system cache that reuses computations for spatio-temporally similar regions. DeepCache [85] enables reuse for closely similar image regions, conducting a single lookup at the input raw images and propagating reusable region boundaries across all application layers to recalculate only the necessary components.

All these solutions have a common aspect: they utilize a software cache that stores a similarity metric of different regions. Some use ad-hoc metrics [13] or features like SIFT [36]. They differ in the search methodology to identify similar neighbor regions. This search activity tends to be very costly, and sometimes, it may outweigh the benefits of computation reuse. They also differ in what they cache: results from image processing subroutines, CNN layer computations, etcetera. In any case, software caching transfers the entire image from the ISP to the main memory and from the main memory to the backend (e.g., CPU), at least to perform the cache lookup, which limits its efficiency and energy-saving potential. On the other hand, our proposal, δ LTA, avoids not only the computation but also the transfer of reusable regions and includes specialized hardware support to minimize the overheads of finding similar regions.

3 DECOUPLING CAMERA SAMPLING

This section presents, δ LTA, our novel solution to overcome the limitations of existing CV systems. We discuss the proposal's core concept and its implementation in modern vision frontends, specifically in the ISP, and describe how to harness its potential using software on the backend CPU. Importantly, any backend, such as a GPU or ASIC, can similarly leverage δ LTA.

3.1 General Overview

δ LTA optimizes the end-to-end image transmission and processing of the CV pipeline. It decouples camera sampling from processing

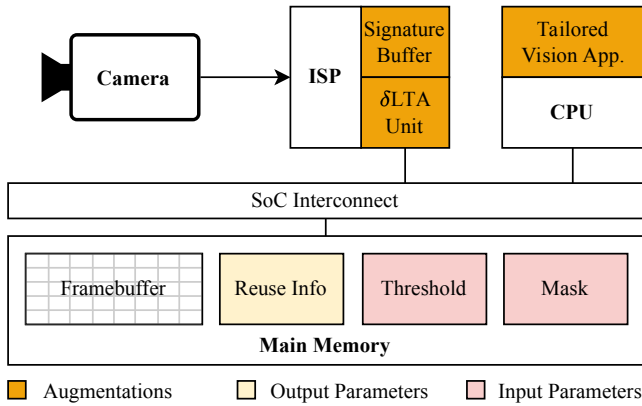


Figure 6: General overview of the proposed vision pipeline.

to avoid unnecessary image data transmissions and redundant computations in the backend. It also enables proactive frontend and backend collaboration.

As Section 2.1 explains, modern CV backends have to access and process all the frames the camera generates, since they cannot make any assumptions about them. Consequently, backend’s workload depends on the frame rate of the camera. δ LTA breaks this coupling, enabling the ISP to discard similar regions in consecutive frames by exploiting the spatio-temporal similarity introduced in Subsection 2.3. Similar regions are not transferred to the main memory nor later accessed by the backend. The backend of a system equipped with δ LTA only has to process the distinctive regions of images that change. For the rest, it can reuse cached results or approximate the output using computationally cheaper methods. δ LTA design transforms a system that processes images based on the camera frame rate to one that processes the images based on how they change.

Moreover, current vision pipelines communicate only in one direction. The frontend processes the images without knowing anything about the requirements of the backend application. Consequently, it fails to adapt to them and always operates the same. For this reason, δ LTA enables proactive communication between both parts of the pipeline, employing a richer interface that informs what image regions changed compared to the previously captured images each time the camera samples. Additionally, it allows the backend to adjust its behavior. The backend can communicate with the frontend to indicate a threshold determining if two regions are similar and a mask to indicate which regions must be sent from the ISP to the Framebuffer (FB) regardless of their similarity. This way, the backend can adjust the performance, accuracy, and energy tradeoffs employing specific runtime information of the vision application.

In summary, we enhance the state-of-the-art vision SoCs with a new ISP capable of filtering similar regions and a new interface that allows the backend to exploit this similarity. Figure 6 depicts the new extensions and frontend interface parameters. In the remainder of this section, we delve into the details of implementing δ LTA and demonstrate its effectiveness in addressing the challenges of efficient and flexible CV processing.

3.2 Frontend

The δ LTA frontend operates by dividing each frame into a grid of equally sized regions, extracting BRIEF [11] descriptors (introduced in Subsection 2.4) that are used as signatures to identify these regions, and comparing incoming frame regions’ signatures with the signatures of the corresponding regions in the FB. If the ISP deems a region similar, the ISP discards it, avoiding its transfer to the FB. Conversely, the ISP transfers the distinct regions to the FB.

Furthermore, we extend the frontend interface to enable proactive collaboration. The new ISP exposes the identifiers of regions that have changed since the previous FB version to the backend in a dedicated *Reuse Information* area within the FB (see Figure 6). The backend can also guide the ISP’s actions by sending parameters via the FB interface, setting the threshold for region similarity detection, and supplying a mask to prevent the ISP from discarding a region, irrespective of the similarity mechanism. This new metadata is vital for facilitating ISP collaboration and allowing the backend to control the ISP’s behavior, adjusting latency, energy, and accuracy tradeoffs on demand according to the requirements of the application, and even enabling the ISP to function as the baseline when required.

We integrated a highly efficient new hardware component into the ISP to support this new functionality that operates in parallel with other ISP stages to hide its latency. This design avoids the BRIEF extraction and matching to increase frame processing latency, even in worst-case scenarios where all regions differ or when the user deactivates δ LTA functionality.

The following subsection presents an example of the δ LTA frontend operation. Next, we will discuss the benefits of using BRIEF descriptors, followed by a description of the microarchitecture of the new component of the δ LTA ISP frontend and its integration within the pipeline.

3.2.1 Operation Overview. Figure 7 provides a high-level overview of δ LTA frontend operation. First, the backend initializes the system with a fixed threshold and an empty mask, indicating to the frontend that it has to send all regions since this is the first frame. The frontend divides the frames into an equally-sized grid. In our example, the ISP splits the frames into a 3×4 grid, with each ISP region having an associated identifier IR_i . Additionally, the color of each region indicates the frame to which it belongs.

The first frame processed in our example is Frame 0. Since it is the first frame, all regions are new, and the ISP must send all of them and their identifiers (Reuse Information) to the FB in the main memory. We refer to the Framebuffer Regions as FR_i . The ISP computes the BRIEF (represented with $\mathcal{B}$ signatures for each IR_i region and stores them in a Signature Buffer (SB). At the end of this process, the FB contains Frame 0 regions (in blue), and the SB stores signatures of every FR_i .

When Frame 1 arrives, the ISP extracts BRIEF signatures for every region and compares them with the corresponding signature in the SB using their Hamming distance. If the Hamming distance of a region is below the threshold indicated by the backend, the ISP classifies it as similar and does not transfer it to the FB (e.g., region IR_1 and IR_N). Otherwise, it sends the region to the FB, updates its BRIEF signature in the SB, and sends the region identifier to the Reuse Information of the frontend interface (e.g., region IR_0). This

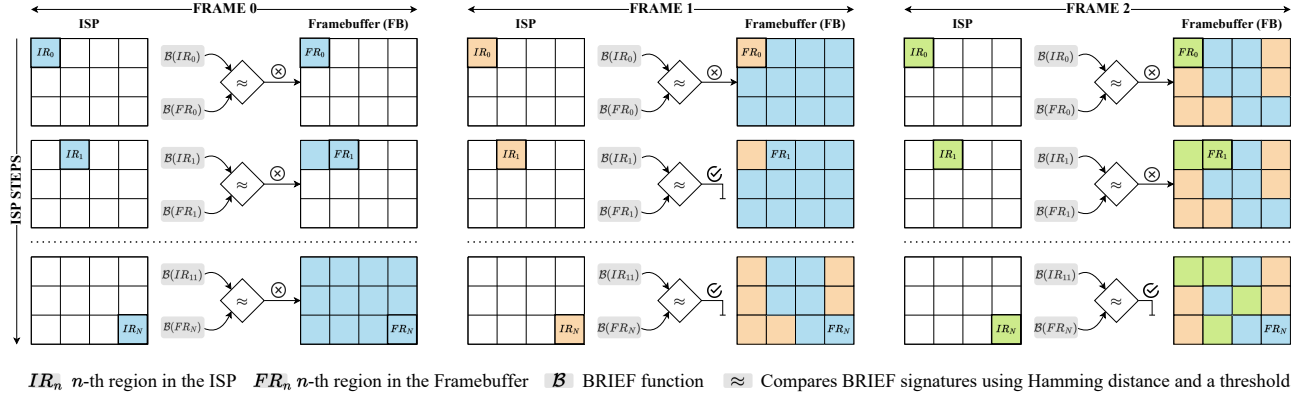


Figure 7: Overview of δ LTA technique for discarding redundant regions in the frontend.

way, the backend knows which regions have changed. At the end of this process, the FB contains regions from Frame 0 (in blue) and Frame 1 (in orange).

When Frame 2 arrives, the same process occurs. The δ LTA ISP processes each region, extracting its BRIEF signature, $B(IR_i)$, and compares it with the one in the SB corresponding to the same region, $B(FR_i)$. It transfers the distinct regions to the FB and their identifiers to the Reuse Information. Upon finishing Frame 2 processing, the ISP sends only four regions, and the FB contains regions from all three processed frames.

3.2.2 Why BRIEF Descriptors? We employ BRIEF [11] as a signature for the grid regions in which δ LTA divides each frame to leverage spatio-temporal frame-to-frame similarity on CV streams. Our decision to utilize BRIEF signatures over other methods [5, 56, 73] in δ LTA is motivated by several key factors:

- **Robustness for δ LTA use-case:** BRIEF signatures can effectively measure similarity between regions under moderate changes like those found in consecutive spatio-temporal similar frames. We do not require scale or rotation invariant features that would raise the hardware complexity and cost.
- **Speed:** BRIEF generation and matching are highly hardware-friendly and computationally efficient, as they only require a limited number of pixel comparisons. The matching consists of computing the Hamming distance between two signatures. Other methods (e.g., SIFT [56]) require costly floating point operations. This efficiency is particularly beneficial in real-time scenarios where speed and energy efficiency are critical.
- **Compact representation:** Storing BRIEF signatures has minimal memory overhead, less than 4% of the image size. This small overhead eliminates the need for the ISP to double buffer past frame contents.

We evaluated the effectiveness of BRIEF signatures to describe the grid regions of frames in CV by conducting experiments to establish the correlation between our method and the Temporal Gradient (TG), already alluded to in Subsection 2.3.

Figure 8a and Figure 8b display the negative correlation between the Normalized TG and the percentage of similar regions detected with BRIEF for 10 and 30 FPS, respectively. The correlation for the 10

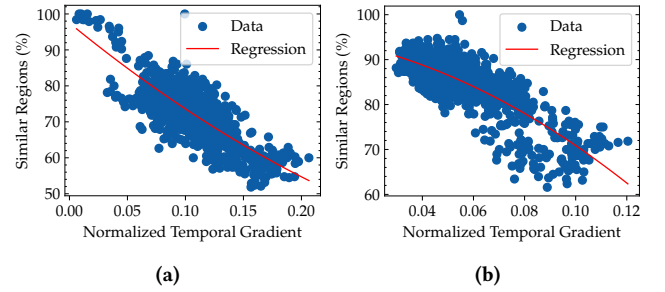


Figure 8: Correlation between the normalized Temporal Gradient and the percentage of similar regions obtained with δ LTA similarity detection method at 10 FPS (8a) and 30 FPS (8b) for the sequence 00 of KITTI [33] benchmark.

FPS case is -0.785, indicating a strong negative relationship between the change between frames (represented by the normalized TG) and the number of similar regions identified by the BRIEF method. A similar correlation (-0.8) exists for the 30 FPS case. This robust negative correlation demonstrates that δ LTA BRIEF-based approach can effectively identify similar regions between consecutive frames. To further support this, we present in Section 5 results showing that our method achieves excellent accuracy when applied to a contemporary localization system.

3.3 ISP Implementation

In this subsection, we describe the microarchitecture implementation of the new δ LTA ISP. As shown in Figure 6, the new hardware includes a δ LTA unit that computes and matches BRIEF signatures of image regions and a Signature Buffer (SB) that stores the BRIEF descriptor of every image region stored in the FB ($B(FR_n)$). Every time the ISP transfers a region, it updates the corresponding signature for the region in the SB to maintain coherence between the SB and the FB contents. Our choice of using BRIEF signatures makes the SB cost relatively low. For example, for an FHD (1920x1080 image) that occupies approximately 2MB, the SB requires only ~64KB.

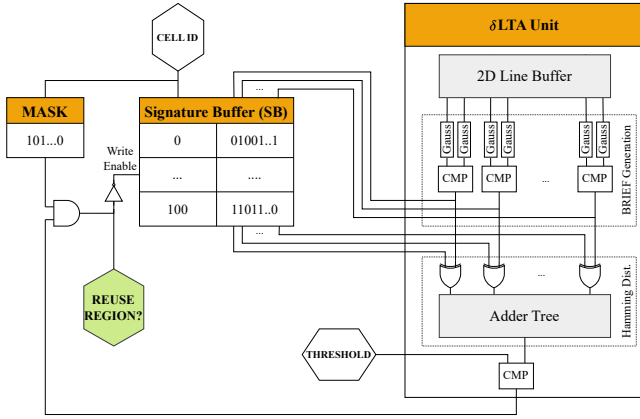


Figure 9: Microarchitecture of the new ISP components: δ LTA unit and Signature Buffer.

3.3.1 δ LTA Unit. Figure 9 illustrates the δ LTA unit microarchitecture, designed for efficient and hardware-friendly filtering of similar image regions. It features a streaming architecture capable of processing one pixel per cycle, utilizing a 2D line buffer to access image regions (IR_n) of 32x32-pixel size. State-of-the-art image processing architectures use streaming line buffers because of their efficiency [22, 79, 80, 82].

The δ LTA unit applies a predefined pattern of pixel intensity comparisons to each specific region to generate its BRIEF signature. Each comparator (CMP), shown in Figure 9, takes Gaussian-filtered pixels from the region as input to generate one bit of the final signature. The *Gauss* modules efficiently perform the filtering using a 3x3 Gaussian kernel approximation, $\frac{1}{16} \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}$, that only requires sums and proper bit-wise wire routing. Filtering of the region data smooths out high-frequency noise and details, improving the robustness of BRIEF.

The unit subsequently compares the newly generated BRIEF signature ($B(IR_n)$) with the stored SB signature from the FB's corresponding region ($B(FR_n)$), calculating the Hamming distance using the sum of bit-wise XOR operations between the two descriptors. Then, it compares the distance with the threshold input parameter for the similarity test and updates the SB if the region is not similar. The mask input parameter determines if the unit updates the region's SB entry, irrespective of the similarity test.

3.3.2 ISP Pipeline Integration. We consider a baseline ISP that resembles literature [9, 38] and commercially available solutions [4, 66, 67]. An ISP sequencer orchestrates various ISP stages to convert the camera sensor's raw data to RGB and YUV color spaces (demosaicing) and to apply several image enhancements afterward. As explained in Section 2.1, some of these improvements adjust the image's color balance to ensure accurate color reproduction (color correction), adjust the gamma and apply High Dynamic Range (HDR) processing to improve contrast and detail in both bright and dark areas (tone mapping) or improve image quality (sharpening), among others [9, 38, 66]. The generated image is temporarily stored in the DMA buffer to smooth data flow within the ISP pipeline before the DMA controller transfers it to the main memory.

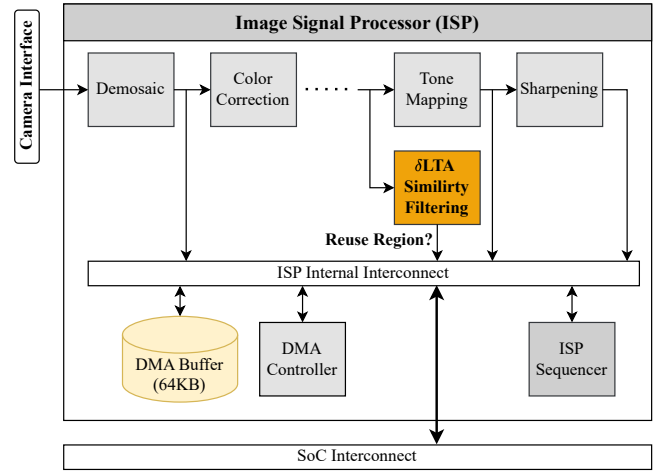


Figure 10: δ LTA incorporates a new ISP stage for BRIEF Similarity Filtering, which overlaps with other ISP stages.

Figure 10 illustrates the described traditional ISP pipeline and the integration of the δ LTA unit into it by adding a new δ LTA Similarity Filtering stage. This stage generates a reuse signal that the DMA controller uses to avoid sending those regions deemed similar by the δ LTA unit.

This new stage runs in parallel with tone mapping and sharpening stages since it does not rely on the images produced after these stages. As described in the previous subsection, the δ LTA unit utilizes a Gaussian filter to blur the regions, effectively negating the image-enhancing advantages of tone mapping and sharpening stages.

We modeled a cycle-accurate RTL hardware description of the δ LTA Similarity Filtering stage and its integration with the two final ISP pipeline stages to demonstrate that the new δ LTA stage latency overlaps with a standard ISP pipeline. We measured that our new stage has a latency of up to 3ms when running at 700 MHz. We confirmed that its latency could overlap even with two simple 5x5 stencil operations. Since the latency of ISP stages in real scenarios is more significant than in our experiments, we can conclude that the δ LTA Similarity Filtering latency can be hidden entirely. This zero-latency overhead design is particularly advantageous for mission-critical real-time visual applications, where minimizing latency is crucial.

Overall, δ LTA frontend allows for highly efficient and hardware-friendly similarity detection, enabling the system to focus only on processing the unique and distinctive regions of the image that change while avoiding redundant computations and unnecessary data transfers. Next, we show an example of how a visual application running in a CPU backend can benefit from using δ LTA.

3.4 Backend

δ LTA frontend is generic and flexible since it makes no assumptions about the application. It divides the image into regions and classifies them based on visual change. On the other hand, the backend can exploit more specialized insights from the application and adapt

Frame 0	Frame 1	Frame 2
C	C	C
C	R	C
C	R	R
C	C	R
C	R	R
C	C	R
C	R	R

Figure 11: Illustration of how the backend CPU reuses the computations for the redundant regions (R) and only processes the non-similar regions (C).

the requirements at runtime, proactively communicating with the frontend through the versatility of our proposed FB interface.

A vision system can exploit δ LTA implementing a background service similar to the ones proposed in prior works [36, 49, 85], but in a more efficient manner. As explained in Subsection 2.5, these previous works leverage frame-to-frame similarity by using image contents as a key for a software cache that temporarily memoizes the associated output results for the regions (e.g., CNN partial convolution results). These methods must access the FB and process the whole image to generate a key (e.g., a descriptor or feature as an identifier) and perform cache lookups for every incoming image. This approach induces an unavoidable energy and latency overhead that reduces the gains for cache hits and increases the worst-case latency since they require reading and processing the entire image for cache lookup. This cost rises with the camera resolution and frame rate increases; hence, the CPU and memory transmission bandwidth limit the benefits of these methods. By leveraging the δ LTA frontend interface, it is possible to mitigate these limitations significantly.

We illustrate the benefits of δ LTA through a crucial emerging application, a state-of-the-art visual localization technique called ORB-SLAM [12, 61] that typically runs on CPUs [30]. We aim to leverage δ LTA to reduce latency and energy consumption without impacting localization accuracy.

To achieve this, we modified the ORB-SLAM codebase to include a software cache to store the ORB features and all its associated metadata (e.g., rBRIEF descriptors) that the algorithm uses as the basis for its entire execution. ORB Feature extraction represents more than 60% of execution time for SLAM and up to 90% for localization [30] as we also reported in Subsection 2.2. By leveraging δ LTA, the modified algorithm only processes the regions that change across consecutive frames while ignoring the remaining ones, as detailed in the following subsections. In Section 5, we demonstrate how the resulting modified version of ORB-SLAM significantly reduces computation and memory requirements while exhibiting the same accuracy.

3.4.1 Operation Overview. The backend initiates the camera pipeline by setting a threshold and an empty blank mask to compute all regions for the first frame. Figure 11 shows the final FB contents seen by the CPU after the δ LTA frontend operation, as discussed in Subsection 3.2.1.

When the δ LTA frontend finishes processing Frame 0, the backend starts processing it. First, the backend reads the Reuse information, which for Frame 0 indicates that all the regions changed

from the last frame (represented with a C in the regions of the figure) since it is the first frame. Based on this information, the backend processes each region, writing the extracted ORB features in a software cache that stores these results for each FR_n .

Upon Frame 1 arrival, the backend repeats the same process. It reads the Reuse Information that informs which FR_n regions changed from the past frame. For these regions, the localization algorithm extracts ORB features as usual, and like in the previous frame, it stores the results in the software cache. The system can reuse the features extracted from the last frame for the rest of the regions, labeled with an R in the figure. The backend does not have access to the pixel values for similar regions, only to the Reuse information. The rest of the algorithm continues the processing, employing the ORB features stored in the cache to localize the camera.

Next, the backend processes Frame 2. It reads the Reuse information and recomputes only the regions that changed from the past FB version. For similar regions, the cache contains the values from either Frame 0 or Frame 1. Similar to what happens with the frontend, where the ISP SB is kept coherent to hold the BRIEF signatures of the regions of the FB, the backend also stores the results (e.g., ORB features) of each region of the FB.

3.4.2 Threshold. A crucial element of our proposal is its adaptability to various application requirements. The threshold parameter, a value between 0 and 1, dictates the intensity with which the frontend discards regions. This threshold can be set on a per-frame basis following different strategies, for example:

- **Static Thresholding:** This approach involves using a fixed threshold throughout the entire execution of the application without any alterations.
- **Utilizing Application-specific Insights:** Many characteristics unique to specific applications can offer valuable information to modify the threshold. For instance, in our case study, the number of matched ORB features or unsuccessful re-localization attempts could be employed to fine-tune the threshold to be more aggressive in regions that have a minor impact on the localization outcome.
- **Multisensor Augmentation:** Additional sensor data from the external environment can be harnessed to adjust the threshold. For example, in some AD systems, the Inertial Measurement Unit (IMU) supplies crucial information on vehicle acceleration [65, 70, 78, 88, 91]. The system can leverage this information to dynamically modify the threshold, lowering it when the vehicle accelerates, turns, or executes unusual maneuvers where heightened accuracy is essential.

For dynamic approaches like the last two above, the system requires a threshold control mechanism receiving feedback from camera pose error estimations. The backend can estimate this error every N frames by conducting two parallel independent pose estimations using the full and δ LTA-filtered frames and comparing the corresponding poses. Our proposal supports this process by masking all the regions and forcing the frontend to send them. Note that the δ LTA frontend still updates the Reuse Information.

Our experiments use conservative static thresholding to illustrate a lower bound of the potential benefits.

3.4.3 Masking Regions. The other mechanism to control δ LTA behavior is a mask. The mask is a binary string where each bit, m_i , tells the ISP if the region IR_i must be sent to the FB regardless of the similarity with previous regions. If the application knows that a region will likely contain important information, it may conservatively decide to recompute it irrespective of the similarity test. Moreover, masking all the regions implies that the frontend operates under baseline conditions, transmitting all regions to the FB. Deactivating δ LTA functionality proves beneficial in conservative situations or where backward compatibility is necessary.

This masking mechanism can be leveraged by drawing insights from the particular visual application. For example, the bounding boxes of objects detected by CNN models exhibit spatial locality in consecutive frames. If the model detected a pedestrian in a frame, it would likely be in the same image region in the next frame. The backend can exploit this mechanism by masking the regions where objects were detected in the precedent frames to force the frontend to send these regions to the FB.

Likewise, in our case study, ORB features in a frame are expected to be in nearby positions in the next frame. Since these features determine the final outcome of the algorithm, the backend could decide to mask the regions that contain a large number of features to force their computation while using a more aggressive threshold for the remaining ones to achieve greater savings without impacting accuracy.

4 EVALUATION METHODOLOGY

We combined different models and simulators to evaluate the continuous vision pipeline's performance, energy, and accuracy. We simulated the entire CV pipeline with gem5 [57] by developing a custom gem5 device to model the ISP behavior. We used DRAMsim3 [48] to model the main memory and the off-the-self CPU models included with gem5 for the CV backend.

We integrated our ISP simulation in gem5 by modeling a device that mimics its framebuffer accesses. We used the Jetson TX2 board's specifications to parametrize the baseline ISP consistent latency and power dissipation characteristics, like other related works[92]. To simulate the specific δ LTA behavior in gem5, we functionally analyzed the input video sequences to forge the Reuse Information generated by the δ LTA ISP. We detected similar regions for each frame using our approach (see Subsection 3.2.1) and the BRIEF implementation of the OpenCV [6] library.

We parallelly modeled the δ LTA Similarity Filtering stage (including the δ LTA unit) to validate its integration within the ISP pipeline using a cycle-accurate RTL hardware description, as explained in Subsection 3.3.2. We ensure the new stage does not create stalls in the ISP pipeline by modeling the streaming pipeline of the last ISP stages (see Figure 10) as two 5x5 stencil operations that are simpler than the typical ISP operations. The architecture incorporates two 2D line buffers for the stencil operations running parallel with our new δ LTA pipeline stage.

Furthermore, the baseline CPU consists of an Out-of-Order (OoO) CPU that resembles an ARM Cortex A72. Our application of interest is ORB-SLAM3 [12, 61] processing the KITTI odometry benchmark [33] sequences. We compare the performance, energy efficiency, and accuracy of three versions of this application:

- **Baseline:** Original version of ORB-SLAM compiled for ARM.
- **Software Caching:** we developed a software caching mechanism (described in Section 2.5) that computes BRIEF signatures of image regions and reuses previously cached results based on their similarity. This mechanism performs cache lookups on the backend CPU, and hence, it needs to read all the image regions from the main memory.
- **δ LTA:** A modified version of ORB-SLAM that incorporates the δ LTA camera interface. This modification enables the reuse of computations from previously processed similar regions, avoiding access to such regions and eliminating software cache lookups.

To evaluate the accuracy, we saved the ORB-SLAM trajectories for all the experiments performed with the three ORB-SLAM versions and used the Evo [35] tool to assess the accuracy using the original ORB-SLAM version trajectories as reference. We also employed an interpolation method based on ffmpeg [25] to increase the FPS of KITTI sequences.

We complemented the performance evaluation with energy consumption and area estimations. We estimated the δ LTA unit power and area using the Synopsys suite [76] and a 14nm cell library to analyze its RTL description. Additionally, we used CACTI [62] to calculate the new BRIEF Signature Buffer (SB) power dissipation. Moreover, we employed DRAMsim3 to get the power estimations of the main memory. For the Backend CPU, we employed McPAT [47] for 32nm using the statistics gathered with gem5. To unify results in a single technology node, we scale the data reported by McPAT and CACTI to 14nm using the methodology described by Stillmaker et al. [75].

Table 1 shows a summary of the main simulation parameters of the baseline platform.

Table 1: Baseline platform simulation parameters.

Component	Properties
ISP	700 MHz, 150mW, 10ms latency
CPU	OoO ARM 64-bit, 3GHz, 3-wide fetch, 8-wide issue 90/256 int/float physical registers, 128-bit NEON vector
Memory	L1: 32KB (I), 2-way + 32KB (D), 2-way; L2: 1 MiB, 16-way; Main Memory: 8GB, DDR3-1600

5 EXPERIMENTAL RESULTS

In this section, we evaluate the effectiveness and efficiency of δ LTA and compare it with a Software Cache (SC) mechanism, a state-of-the-art approach introduced in Subsection 2.5 (i.e., Deepmon [40]).

In our localization experiments, we utilize a static threshold established before execution. We have modified ORB-SLAM to disable SC and δ LTA during system initialization, relocation, or tracking loss. Furthermore, we conservatively mask regions that contained at least one feature in the preceding frame.

We use the terms SC-TH and δ LTA-TH to refer to the SC or the δ LTA scheme with a threshold value of TH. As for benchmarks, we use all sequences of the KITTI dataset.

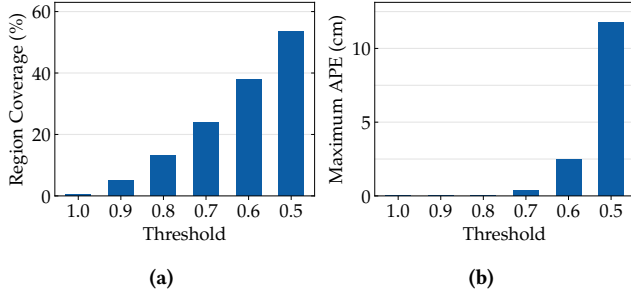


Figure 12: (12a) Percentage of similar regions. (12b) Maximum Absolute Pose Error (APE) for different thresholds.

5.1 Region Coverage and Accuracy Analysis

Figure 12a presents the average percentage of regions our method deems similar while the localization application processes the KITTI dataset. The lower the threshold, the higher the percentage of similar regions. This increased region coverage not only enhances the efficiency of the system but also enables it to process more relevant data in real-time.

On the other hand, it is crucial to maintain the accuracy of the application. We assessed the localization accuracy compared to the Baseline using the Absolute Pose Error (APE). APE serves as a key performance metric for localization algorithms, measuring the difference between the ground truth pose and the estimated pose at each time step. High localization accuracy is essential, particularly for AD systems that require centimeter-level precision. Consequently, we report the maximum APE as our central accuracy metric since a single incorrect critical estimation can significantly impact the user experience. Figure 12b presents the maximum APE for all KITTI sequences. We can see that our method introduces no error for threshold values up to 0.7. For 0.7, there are no errors except for sequence 02, which incurs a minor error of half a centimeter in the worst case. Higher thresholds begin to incur some non-negligible errors, so we conclude that 0.7 is the most appropriate threshold and is the one adopted by our experiments unless otherwise indicated.

5.2 Latency Analysis

To evaluate the performance of our proposed solution, we examine the end-to-end latency and the 99th percentile tail latency of the CV pipeline, excluding the image sensor. Tail latency is paramount in the context of AD systems since they have extreme constraints regarding response time. Figure 13a illustrates the normalized average latency, comparing the Baseline, SC, and δ LTA. Figure 13b depicts the tail latency. To measure the overheads of the different approaches, we also evaluate both δ LTA and SC with a threshold of 1, representing a scenario where minimal region reuse occurs (approximately 0.53%, when the camera stops). In this case, the benefits offered by these schemes are negligible.

Our experiments led to several conclusions. SC's benefits are minor in average latency and practically null in tail latency when not harming it (SC-1, 0.9, and 0.8). The overhead of cache maintenance and lookups offsets the reuse benefits, leading to a negative

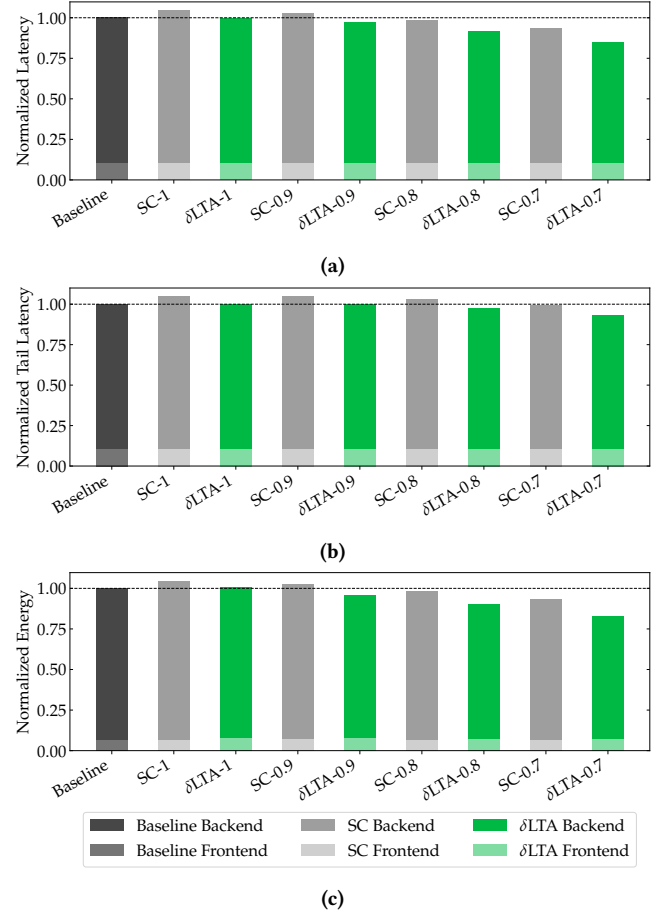


Figure 13: Comparison of Normalized Latencies (13a), Normalized Tail Latencies (13b), and Normalized Energy (13c) for Baseline, Software Caching (SC), and δ LTA.

impact on the worst-case response of the system, which in turn can introduce safety concerns in mission-critical scenarios such as AD.

On the other hand, δ LTA introduces no overhead since other ISP tasks entirely hide its latency. Our proposal greatly outperforms SC mechanisms in both average and tail latency reduction. It achieves an average latency reduction of 15.22% and a tail latency reduction of 7.2% over the Baseline, as well as an average latency reduction of 9.15% and a tail latency reduction of 7% over SC.

5.3 Energy Analysis

Figure 13c compares energy consumption among the Baseline, SC, and δ LTA configurations. δ LTA achieves a 17% reduction in energy consumption compared to the Baseline and a 11% reduction compared to the SC scheme. These results emphasize the advantages of decoupling camera sampling from processing. The additional energy savings of δ LTA come from fewer ISP and CPU FB accesses and a more energy-efficient implementation of similarity detection.

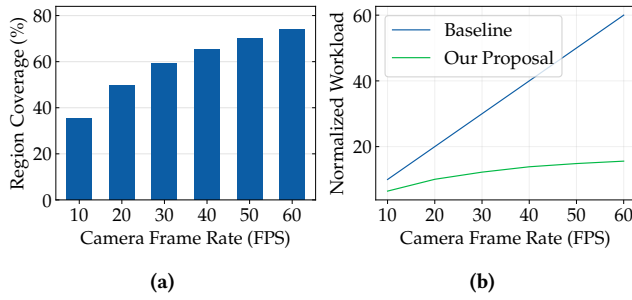


Figure 14: (14a) Percentage of region coverage for different camera rates. (14b) Amount of computations required when increasing frame rate.

5.4 Frame Rate Scaling Analysis

In this section, we examine a central advantage of δ LTA when processing higher camera frame rates, something to expect in future systems. Figure 14a illustrates how frame-to-frame similarity in a given scene increases as the frame rate ascends (without masking regions). Increasing the frame rate reduces the time between samples, leading to fewer changes and, thus, more similarity.

We study the backend workload response of our proposal when scaling up the frame rate and compare it to Baseline. We assume ideal conditions, meaning no ISP, main memory, or backend CPU bottlenecks, allowing the system to process arbitrary amounts of pixels. Figure 14b shows the results of this sensitivity experiment.

In the Baseline system without δ LTA, the backend must access all image pixels, resulting in an increased workload as the camera frame rate grows (see blue line in Figure 14b). While a SC mechanism can assist the system in skipping some computations to exploit the similarity increase, it does not eliminate the need for processing the images. In more realistic scenarios, the inherent overhead of the SC would eventually become dominant, hindering the system's ability to function and even resolve SC hits in real-time. Thus, the Baseline backend workload depends on the camera frame rate, and it not only constrains the system's real-time responsiveness but also misses potential opportunities for energy savings.

In contrast, the δ LTA-equipped system discards redundant image regions based on similarity, which for a given scene becomes more frequent as the frame rate increases. Beyond a certain point, more samples do not provide more relevant information about the scene; therefore, the number of distinct regions the backend needs to process eventually stagnates (see green line in Figure 14b). As demonstrated, our proposal enables CV SoC to break the backend workload dependency from camera sampling, allowing it to adjust the processing rate based on the amount of information in the captured scene.

5.5 Area and Power Analysis

We assess the area and power costs of δ LTA using the approach described in Section 4, considering that the δ LTA unit supports up to Full HD resolution (standard nowadays).

δ LTA introduces a minor area overhead of $0.122mm^2$ due to the ISP augmentations. The main area overheads come from the $\sim 64KB$ for the Signature Buffer to store BRIEF descriptors and the $\sim 60KB$

for the 2D line buffer. This area overhead is negligible compared to a typical SoC area, such as the $100mm^2$ of a Mediatek A72 cluster [59], the $88mm^2$ of Apple A14 [83], or the $350mm^2$ of Nvidia Xavier[64]. δ LTA dissipates 50mW mainly due to the 2D Line Buffer that needs to read and write several pixels during ISP processing.

6 RELATED WORK

Improving the performance of CV systems has attracted the research community's attention in recent years. Localization, object recognition, and object tracking represent three major active research areas for visual applications.

Feature-based localization methods have gained significant attention in autonomous machines, robotics, and Augmented Reality/Virtual Reality applications [27, 45, 77, 88]. Key developments include localization/registration [29], Visual-Inertial Odometry (VIO) [70], and Simultaneous Localization and Mapping (SLAM) [12, 24, 26, 61]. The recently published SOFT2 [19] visual SLAM system currently ranks at the top for the KITTI [33] dataset, surpassing even high-cost lidar-based alternatives. Its great performance motivated our choice of a visual localization application to demonstrate the advantages of δ LTA.

Substantial research has leveraged deep learning for mobile object recognition and object tracking. Multiple optimizations have been proposed, including the use of simplified models [39, 71], model compression techniques [18, 37, 60], quantization methods [34, 43], and specialized hardware support [1, 15, 44, 68, 74].

Like δ LTA, several prior developments have capitalized on temporal and spatial redundancies in real-time frames to reduce computational complexity. These approaches can be divided into software and hardware categories.

Numerous software-based solutions function as a service, acting as a cache to minimize system-wide computational redundancy in concurrent vision applications, as described in Section 2.5. In contrast to these previous works, our approach offloads similarity computation and matching to the ISP rather than relying on CPU/IP calculations. We proposed an efficient similarity detection overlapped with other ISP stages that, unlike other works, ensures that the overhead of δ LTA remains minimal even in the worst-case scenario.

Some previous innovative hardware solutions also capitalize on temporal/spatial redundancies in real-time frames to streamline computations on mobile CPUs [23, 58, 72]. PVF [31] harnesses the predictability of CV streams to introduce a speculative vision pipeline. This pipeline allows the frontend to predict future frames, scheduling them speculatively into the SoC's heterogeneous IPs. When the actual frame arrives, the systems assess the predicted and actual frames similarly; if similar, the corresponding IP processing of the predicted frame may have already finished, minimizing frame latency. While our scheme decouples camera sampling, PVF decouples frontend processing from vision computation through speculation. PVF can be seamlessly integrated with our frontend concept by predicting frames and speculatively processing only distinct regions, estimated with our scheme. Other proposals exploit motion vectors derived from classical optical flow methods [8] or hardware codecs [87] for CNN inference for object detection and

tracking. These approaches can be merged with our novel frontend interface to perform partial inference and avoid redundant processing.

Euphrates [92] employ motion information collected from high-end ISPs to bypass CNN inference for specific frames. The system performs full inferences of a subset of frames and uses the motion vectors extracted from the ISP to predict the results for the intermediate ones. Specifically, this technique translates the bounding boxes to their new positions based on the direction and magnitude of the perceived motion. However, this approach faces a critical issue: new objects entering the scene during frames with motion vector updates might go undetected. Combining Euphrates with our technique can overcome this limitation. For intermediate frames, if δ LTA frontend identifies a significantly changed region distant from a previously detected object, it could mean that a new entity entered the scene, which must trigger a full inference. This combined technique could use motion vectors to estimate results for the δ LTA-detected distinct regions and reuse previously cached computations for the rest. Euphrates relies on high-end ISPs with costly motion denoising, which can increase the frontend's cost and power dissipation. Some works suggest using simplified ISPs or working directly with RAW image data for computer vision [9, 21]. Unlike these previous works, our approach is versatile, imposing no assumptions on the data; it can be adapted to processing RAW data or integrated into a simpler ISP with small hardware extensions.

Autonomous driving represents a highly active research area where real-time, accurate, and energy-efficient vision systems are crucial and challenging [46, 51, 52, 89]. Innumerable solutions have been proposed in recent years to optimize the feature-based localization discussed in this work, including ASICs [79, 80], FPGAs [53], frameworks for automatically generating synthesizable accelerators [55], and accelerator-host communication approaches [78]. Another vital research direction involves multi-sensor fusion, such as camera-IMU fusion [30, 91]. All these prior solutions are complementary and can be enhanced when integrated with our proposal, optimizing their computations through the δ LTA frontend. On the other hand, our work could be extended by leveraging IMU data to fine-tune the threshold used for similarity computations.

Framebuffer compression is another related technique. Industrial examples include ARM AFBC [3], Imagination IMGIC [41], AMD DCC [7], and Chips&Media CFrame [17]. These proprietary techniques require decompression before processing by the consumer IP, implying that they do not discard redundancies or communicate with the IP about them. Compression reduces the energy consumption of the frame transmission but does not decouple the frontend from the backend like δ LTA does. Besides, delta encoding lossless compression, one of the most popular industry choices for these techniques [7, 69], can straightforwardly complement δ LTA by compressing the distinct regions detected by the ISP.

In summary, while other techniques focus on specific use cases or localized optimizations, our approach offers broad versatility, integrating with both existing and new CV solutions. δ LTA uniquely exploits CV pipeline optimization opportunities by harnessing frame-to-frame similarity and innovatively isolating the backend processing from the camera sampling rate. Our scheme's efficiency and flexibility position it as a significant forward step in CV SoC design.

7 CONCLUSIONS

In this paper, we proposed δ LTA (δ ont't Look Twice, it's Alright), a novel frontend that decouples the camera frame rate from the backend processing rate by augmenting the camera with the ability to discard redundant image regions while notifying the backend of the specific image areas that exhibit substantial changes compared to the preceding ones.

Our evaluation shows that our proposal can significantly reduce the data sent downstream while maintaining high accuracy. Our approach reduces the application latency by 15.22% and its energy consumption by 17%, significantly improving the performance of CV pipelines even in worst-case scenarios. We also show that δ LTA enables systems to process higher sampling frame rates by focusing solely on processing the valuable visual information, which enhances general responsiveness. Consequently, solutions like δ LTA are notably promising for emerging critical applications such as AD or AR/VR. Finally, our work also exemplifies the potential for optimizations in modern vision pipelines, including the ISP, when carefully considering its holistic end-to-end operation perspective.

ACKNOWLEDGMENTS

This work has been supported by the CoCoUnit ERC Advanced Grant of the EU's Horizon 2020 program (grant No 833057), the Spanish State Research Agency (MCIN/AEI) under grant PID2020-113172RB-I00, the ICREA Academia program, and the FPU grant FPU18/04413.

REFERENCES

- [1] Jorge Albericio, Patrick Judd, Tayler Hetherington, Tor Aamodt, Natalie Enright Jerger, and Andreas Moshovos. 2016. Cnvlutin: Ineffectual-Neuron-Free Deep Neural Network Computing. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. 1–13. <https://doi.org/10.1109/ISCA.2016.11> ISSN: 1063-6897.
- [2] Sergei Alyamkin, Matthew Ardi, Alexander C. Berg, Achille Brighton, Bo Chen, Yiran Chen, Hsin-Pai Cheng, Zichen Fan, Chen Feng, Bo Fu, Kent Gauen, Abhinav Goel, Alexander Goncharenko, Xuyang Guo, Soonhoi Ha, Andrew Howard, Xiao Hu, Yuanjun Huang, Donghyun Kang, Jaeyoun Kim, Jong Gook Ko, Alexander Kondratyev, Junhyeok Lee, Seungjae Lee, Suwoong Lee, Zichao Li, Zhiyu Liang, Juzheng Liu, Xin Liu, Yang Lu, Yung-Hsiang Lu, Deeptanshu Malik, Hong Hanh Nguyen, Eunbyung Park, Denis Repin, Liang Shen, Tao Sheng, Fei Sun, David Svitov, George K. Thiruvathukal, Baiwu Zhang, Jingchi Zhang, Xiaopeng Zhang, and Shaojie Zhuo. 2019. Low-Power Computer Vision: Status, Challenges, and Opportunities. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 9, 2 (June 2019), 411–421. <https://doi.org/10.1109/JETCAS.2019.2911899> Conference Name: IEEE Journal on Emerging and Selected Topics in Circuits and Systems.
- [3] Arm. 2022. *Arm Frame Buffer Compression*. <https://www.arm.com/technologies/graphics-technologies/arm-frame-buffer-compression>
- [4] Arm. 2023. *Mali-C32*. <https://developer.arm.com/Processors/Mali-C32>
- [5] Herbert Bay, Tinne Tuytelaars, and Luc Van Gool. 2006. SURF: Speeded Up Robust Features. In *Computer Vision – ECCV 2006 (Lecture Notes in Computer Science)*, Aleš Leonardis, Horst Bischof, and Axel Pinz (Eds.). Springer, Berlin, Heidelberg, 404–417. https://doi.org/10.1007/11744023_32
- [6] G. Bradski. 2000. The OpenCV Library. *Dr. Dobbs's Journal of Software Tools* (2000).
- [7] Chris Brennan. 2016. *Getting the Most Out of Delta Color Compression*. <https://gpuopen.com/learn/dcc-overview/>
- [8] Mark Buckler, Philip Bedoukian, Suren Jayasuriya, and Adrian Sampson. 2018. EVA²: Exploiting Temporal Redundancy in Live Computer Vision. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. 533–546. <https://doi.org/10.1109/ISCA.2018.00051> ISSN: 2575-713X.
- [9] Mark Buckler, Suren Jayasuriya, and Adrian Sampson. 2017. Reconfiguring the Imaging Pipeline for Computer Vision. In *2017 IEEE International Conference on Computer Vision (ICCV)*. 975–984. <https://doi.org/10.1109/ICCV.2017.111> ISSN: 2380-7504.
- [10] Mihai Bujanca, Xuesong Shi, Matthew Spear, Pengpeng Zhao, Barry Lennox, and Mikel Luján. 2021. Robust SLAM Systems: Are We There Yet?. In *2021 IEEE/RSJ*

- International Conference on Intelligent Robots and Systems (IROS)*. IEEE Press, Prague, Czech Republic, 5320–5327. <https://doi.org/10.1109/IROS51168.2021.9636814>
- [11] Michael Calonder, Vincent Lepetit, Christoph Strecha, and Pascal Fua. 2010. Brief: Binary robust independent elementary features. In *European conference on computer vision*. Springer, 778–792.
- [12] Carlos Campos, Richard Elvira, Juan J. Gómez Rodríguez, José M. M. Montiel, and Juan D. Tardós. 2021. ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual–Inertial, and Multimap SLAM. *IEEE Transactions on Robotics* 37, 6 (Dec. 2021), 1874–1890. <https://doi.org/10.1109/TRO.2021.3075644> Conference Name: IEEE Transactions on Robotics.
- [13] Lukas Cavigelli, Philippe Degen, and Luca Benini. 2017. CBInfer: Change-Based Inference for Convolutional Neural Networks on Video Data. In *Proceedings of the 11th International Conference on Distributed Smart Cameras (ICDSC 2017)*. Association for Computing Machinery, New York, NY, USA, 1–8. <https://doi.org/10.1145/3131885.3131906>
- [14] Dimitris Chatzopoulos, Carlos Bermejo, Zhanpeng Huang, and Pan Hui. 2017. Mobile Augmented Reality Survey: From Where We Are to Where We Go. *IEEE Access* 5 (2017), 6917–6950. <https://doi.org/10.1109/ACCESS.2017.2698164> Conference Name: IEEE Access.
- [15] Yu-Hsin Chen, Tushar Krishna, Joel Emer, and Vivienne Sze. 2016. 14.5 Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. In *2016 IEEE International Solid-State Circuits Conference (ISSCC)*. 262–263. <https://doi.org/10.1109/ISSCC.2016.7418007> ISSN: 2376-8606.
- [16] Jun Cheng, Liyan Zhang, Qihong Chen, Xinrong Hu, and Jingcao Cai. 2022. A review of visual SLAM methods for autonomous driving vehicles. *Engineering Applications of Artificial Intelligence* 114 (Sept. 2022), 104992. <https://doi.org/10.1016/j.engappai.2022.104992>
- [17] Chips&Media, Inc. 2023. *MAPI Image Processing IP for Pre/Post Processing*. <https://www.chipsmedia.com/mapi>
- [18] Yoojin Choi, Mostafa El-Khamy, and Jungwon Lee. 2020. Universal Deep Neural Network Compression. *IEEE Journal of Selected Topics in Signal Processing* 14, 4 (May 2020), 715–726. <https://doi.org/10.1109/JSTSP.2020.2975903> Conference Name: IEEE Journal of Selected Topics in Signal Processing.
- [19] Igor Cvišić, Ivan Marković, and Ivan Petrović. 2022. SOFT2: Stereo Visual Odometry for Road Vehicles Based on a Point-to-Epipolar-Line Metric. *IEEE Transactions on Robotics* (2022). Publisher: IEEE.
- [20] Klaus David and Alexander Flach. 2010. CAR-2-X and Pedestrian Safety. *IEEE Vehicular Technology Magazine* 5, 1 (March 2010), 70–76. <https://doi.org/10.1109/MVT.2009.935536> Conference Name: IEEE Vehicular Technology Magazine.
- [21] Zidong Du, Robert Fasthuber, Tianshi Chen, Paolo Ienne, Ling Li, Tao Luo, Xiaobing Feng, Yunji Chen, and Olivier Temam. 2015. ShiDianNao: Shifting vision processing closer to the sensor. In *2015 ACM/IEEE 42nd Annual International Symposium on Computer Architecture (ISCA)*. 92–104. <https://doi.org/10.1145/2749469.2750389> ISSN: 1063-6897.
- [22] Weikang Fang, Yanjun Zhang, Bo Yu, and Shaoshan Liu. 2017. FPGA-based ORB feature extraction for real-time visual SLAM. In *2017 International Conference on Field Programmable Technology (ICFPT)*. IEEE, 275–278.
- [23] Yu Feng, Paul Whatmough, and Yuhao Zhu. 2019. ASV: Accelerated Stereo Vision System. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 643–656. <https://doi.org/10.1145/3352460.3358253>
- [24] Maxime Ferrera, Alexandre Eudes, Julien Moras, Martial Sanfourche, and Guy Le Besnerais. 2021. OV²SLAM: A Fully Online and Versatile Visual SLAM for Real-Time Applications. *IEEE Robotics and Automation Letters* 6, 2 (April 2021), 1399–1406. <https://doi.org/10.1109/LRA.2021.3058069> Conference Name: IEEE Robotics and Automation Letters.
- [25] FFmpeg Developers. 2023. *FFmpeg tool (Version n6.0)*. <http://ffmpeg.org/>
- [26] Qiang Fu, Hongshan Yu, Xiaolong Wang, Zhengeng Yang, Yong He, Hong Zhang, and Ajmal Mian. 2022. Fast ORB-SLAM Without Keypoint Descriptors. *IEEE Transactions on Image Processing* 31 (2022), 1433–1446. <https://doi.org/10.1109/TIP.2021.3136710> Conference Name: IEEE Transactions on Image Processing.
- [27] Jorge Fuentes-Pacheco, José Ruiz-Ascencio, and Juan Manuel Rendón-Mancha. 2015. Visual simultaneous localization and mapping: a survey. *Artificial Intelligence Review* 43, 1 (2015), 55–81. Publisher: Springer.
- [28] Hamed Kiani Galoogahi, Ashton Fagg, Chen Huang, Deva Ramanan, and Simon Lucey. 2017. Need for Speed: A Benchmark for Higher Frame Rate Object Tracking. In *2017 IEEE International Conference on Computer Vision (ICCV)*. 1134–1143. <https://doi.org/10.1109/ICCV.2017.128> ISSN: 2380-7504.
- [29] Dorian Galvez-López and Juan D. Tardós. 2012. Bags of Binary Words for Fast Place Recognition in Image Sequences. *IEEE Transactions on Robotics* 28, 5 (Oct. 2012), 1188–1197. <https://doi.org/10.1109/TRO.2012.2197158> Conference Name: IEEE Transactions on Robotics.
- [30] Yiming Gan, Yu Bo, Boyuan Tian, Leimeng Xu, Wei Hu, Shaoshan Liu, Qiang Liu, Yanjun Zhang, Jie Tang, and Yuhao Zhu. 2021. Eudoxus: Characterizing and accelerating localization in autonomous machines industry track paper. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 827–840.
- [31] Yiming Gan, Yuxian Qiu, Lele Chen, Jingwen Leng, and Yuhao Zhu. 2020. Low-Latency Proactive Continuous Vision. In *Proceedings of the ACM International Conference on Parallel Architectures and Compilation Techniques (PACT '20)*. Association for Computing Machinery, New York, NY, USA, 329–342. <https://doi.org/10.1145/3410463.3414650>
- [32] Boyu Gao, Haoxiang Lang, and Jing Ren. 2020. Stereo Visual SLAM for Autonomous Vehicles: A Review. In *2020 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. IEEE, 1316–1322.
- [33] Andreas Geiger, Philip Lenz, and Raquel Urtasun. 2012. Are we ready for autonomous driving? the kitti vision benchmark suite. In *2012 IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 3354–3361.
- [34] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. 2021. A Survey of Quantization Methods for Efficient Neural Network Inference. <https://doi.org/10.48550/arXiv.2103.13630> arXiv:2103.13630 [cs].
- [35] Michael Grupp. 2017. evo: Python package for the evaluation of odometry and SLAM. <https://github.com/MichaelGrupp/evo>
- [36] Peizhen Guo and Wenjun Hu. 2018. Potluck: Cross-Application Approximate Deduplication for Computation-Intensive Mobile Applications. In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '18)*. Association for Computing Machinery, New York, NY, USA, 271–284. <https://doi.org/10.1145/3173162.3173185>
- [37] Song Han, Huizi Mao, and William J. Dally. 2016. Deep Compression: Compressing Deep Neural Network with Pruning, Trained Quantization and Huffman Coding. In *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2–4, 2016, Conference Track Proceedings*, Yoshua Bengio and Yann LeCun (Eds.). <http://arxiv.org/abs/1510.00149>
- [38] James Hegarty, John Brunhaver, Zachary DeVito, Jonathan Ragan-Kelley, Noy Cohen, Steven Bell, Artem Vasilyev, Mark Horowitz, and Pat Hanrahan. 2014. Darkroom: compiling high-level image processing code into hardware pipelines. *ACM Transactions on Graphics* 33, 4 (July 2014), 144:1–144:11. <https://doi.org/10.1145/2601097.2601174>
- [39] Andrew Howard, Mark Sandler, Bo Chen, Weijun Wang, Liang-Chieh Chen, Mingxing Tan, Grace Chu, Vijay Vasudevan, Yukun Zhu, Ruoming Pang, Hartwig Adam, and Quoc Le. 2019. Searching for MobileNetV3. *IEEE Computer Society*, 1314–1324. <https://doi.org/10.1109/ICCV.2019.00140>
- [40] Loc N. Huynh, Youngki Lee, and Rajesh Krishna Balan. 2017. DeepMon: Mobile GPU-based Deep Learning Framework for Continuous Vision Applications. In *Proceedings of the 15th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys '17)*. Association for Computing Machinery, New York, NY, USA, 82–95. <https://doi.org/10.1145/3081333.3081360>
- [41] Imagination. 2022. *IMGIC Technology*. <https://www.imaginationtech.com/resources/imgic-technology/>
- [42] Idaku Ishii, Tetsuro Tatebe, Qingyi Gu, Yuta Morie, Takeshi Takaki, and Kenji Tajima. 2010. 2000 fps real-time vision system with high-frame-rate video recording. In *2010 IEEE International Conference on Robotics and Automation*. 1536–1541. <https://doi.org/10.1109/ROBOT.2010.5509731> ISSN: 1050-4729.
- [43] Shubham Jain, Swagath Venkataramani, Vijayalakshmi Srinivasan, Jungwook Choi, Pierce Chuang, and Leland Chang. 2018. Compensated-DNN: energy efficient low-precision deep neural networks by compensating quantization errors. In *Proceedings of the 55th Annual Design Automation Conference (DAC '18)*. Association for Computing Machinery, New York, NY, USA, 1–6. <https://doi.org/10.1145/3195970.3196012>
- [44] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre-luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley, Matt Dau, Jeffrey Dean, Ben Gelb, Tara Vazir Ghaemmaghami, Rajendra Gottipati, William Gulland, Robert Hagmann, C. Richard Ho, Doug Hogberg, John Hu, Robert Hundt, Dan Hurt, Julian Ibarz, Aaron Jaffey, Alek Jaworski, Alexander Kaplan, Harshit Khaitan, Daniel Killebrew, Andy Koch, Naveen Kumar, Steve Lacy, James Laudon, James Law, Diemthu Le, Chris Leary, Zhuyuan Liu, Kyle Lucke, Alan Lundin, Gordon MacKean, Adriana Maggiore, Maire Mahony, Kieran Miller, Rahul Nagarajan, Ravi Narayanaswami, Ray Ni, Kathy Nix, Thomas Norrie, Mark Omernick, Narayana Penukonda, Andy Phelps, Jonathan Ross, Matt Ross, Amir Salek, Emad Samadiani, Chris Severn, Gregory Sizikov, Matthew Snelham, Jed Souter, Dan Steinberg, Andy Swing, Mercedes Tan, Gregory Thorson, Bo Tian, Horia Toma, Erick Tuttle, Vijay Vasudevan, Richard Walter, Walter Wang, Eric Wilcox, and Doe Hyun Yoon. 2017. In-Datacenter Performance Analysis of a Tensor Processing Unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA '17)*. Association for Computing Machinery, New York, NY, USA, 1–12. <https://doi.org/10.1145/3079856.3080246>
- [45] Iman Abaspur Kazerouni, Luke Fitzgerald, Gerard Dooly, and Daniel Toal. 2022. A Survey of State-of-the-Art on Visual SLAM. *Expert Systems with Applications* (2022), 117734. Publisher: Elsevier.
- [46] Srivatsan Krishnan, Zishen Wan, Kshitij Bhardwaj, Paul Whatmough, Aleksandra Faust, Sabrina Neuman, Gu-Yeon Wei, David Brooks, and Vijay Janapa Reddi.

2022. Automatic Domain-Specific SoC Design for Autonomous Unmanned Aerial Vehicles. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 300–317. <https://doi.org/10.1109/MICRO56248.2022.00033>
- [47] Sheng Li, Jung Ho Ahn, Richard D Strong, Jay B Brockman, Dean M Tullsen, and Norman P Jouppi. 2009. McPAT: An integrated power, area, and timing modeling framework for multicore and manycore architectures. In *Proceedings of the 42nd annual IEEE/ACM international symposium on microarchitecture*. 469–480.
- [48] Shang Li, Zhiyuan Yang, Dhiraj Reddy, Ankur Srivastava, and Bruce Jacob. 2020. DRAMsim3: A Cycle-Accurate, Thermal-Capable DRAM Simulator. *IEEE Computer Architecture Letters* 19, 2 (July 2020), 106–109. <https://doi.org/10.1109/LCA.2020.2973991> Conference Name: IEEE Computer Architecture Letters.
- [49] Robert LiKamWa and Lin Zhong. 2015. Starfish: Efficient Concurrency Support for Computer Vision Applications. In *Proceedings of the 13th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys '15)*. Association for Computing Machinery, New York, NY, USA, 213–226. <https://doi.org/10.1145/2742647.2742663>
- [50] Kysam Lim, Gye Su Kim, Suki Kim, and Kwang-Hyun Baek. 2010. A multi-lane MIPI CSI receiver for mobile camera applications. *IEEE Transactions on Consumer Electronics* 56, 3 (Aug. 2010), 1185–1190. <https://doi.org/10.1109/TCE.2010.5606244> Conference Name: IEEE Transactions on Consumer Electronics.
- [51] Shih-Chieh Lin, Yunqi Zhang, Chang-Hong Hsu, Matt Skach, Md E. Haque, Lingjia Tang, and Jason Mars. 2018. The Architectural Implications of Autonomous Driving. *ACM SIGPLAN Notices* 53, 2 (March 2018), 751–766. <https://doi.org/10.1145/3296957.3173191> Publisher: Association for Computing Machinery (ACM).
- [52] Liangkai Liu, Sidi Lu, Ren Zhong, Wu Baofu, Yongtao Yao, Qingyang Zhang, and Weisong Shi. 2020. Computing Systems for Autonomous Driving: State-of-the-Art and Challenges. *IEEE Internet of Things Journal* PP (Dec. 2020), 1–1. <https://doi.org/10.1109/IJOT.2020.3043716>
- [53] Runze Liu, Jianlei Yang, Yiran Chen, and Weisheng Zhao. 2019. ESLAM: An energy-efficient accelerator for real-time ORB-SLAM on FPGA platform. In *Proceedings of the 56th Annual Design Automation Conference 2019*. 1–6.
- [54] Shaoshan Liu, Liangkai Liu, Jie Tang, Bo Yu, Yifan Wang, and Weisong Shi. 2019. Edge Computing for Autonomous Driving: Opportunities and Challenges. *Proc. IEEE* 107, 8 (Aug. 2019), 1697–1716. <https://doi.org/10.1109/JPROC.2019.2915983> Conference Name: Proceedings of the IEEE.
- [55] Weizhuang Liu, Bo Yu, Yiming Gan, Qiang Liu, Jie Tang, Shaoshan Liu, and Yuhao Zhu. 2021. Archytas: A Framework for Synthesizing and Dynamically Optimizing Accelerators for Robotic Localization. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '21)*. Association for Computing Machinery, New York, NY, USA, 479–493. <https://doi.org/10.1145/3466752.3480077>
- [56] D.G. Lowe. 1999. Object recognition from local scale-invariant features. In *Proceedings of the Seventh IEEE International Conference on Computer Vision*, Vol. 2. 1150–1157 vol.2. <https://doi.org/10.1109/ICCV.1999.790410>
- [57] Jason Lowe-Power, Abdul Mutaal Ahmad, Ayaz Akram, Mohammad Alian, Rico Amslinger, Matteo Andreozzi, Adrià Arnejach, Nils Asmussen, Brad Beckmann, Srikant Bharadwaj, and others. 2020. The gem5 simulator: Version 20.0+. *arXiv preprint arXiv:2007.03152* (2020).
- [58] Mostafa Mahmoud, Kevin Siu, and Andreas Moshovos. 2018. Diffy: a Déjà vu-Free Differential Deep Neural Network Accelerator. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 134–147. <https://doi.org/10.1109/MICRO.2018.00020>
- [59] Hugh T Mair, Gordon Gammie, Alice Wang, Rolf Lagerquist, C.J. Chung, Sumanth Gururajaro, Ping Kao, Anand Rajagopalan, Anirban Saha, Amit Jain, Ericbill Wang, Shichin Ouyang, Huajun Wen, Achuta Thippa, HsinChen Chen, Syed Rahman, Minh Chau, Anshul Varma, Brian Flachs, Mark Peng, Alfred Tsai, Vincent Lin, Ue Fu, Wuan Kuo, Lee-Kee Yong, Clavin Peng, Leo Shieh, Jengding Wu, and Uming Ko. 2016. 4.3 A 20nm 2.5GHz ultra-low-power tri-cluster CPU subsystem with adaptive power allocation for optimal mobile SoC performance. In *2016 IEEE International Solid-State Circuits Conference (ISSCC)*. 76–77. <https://doi.org/10.1109/ISSCC.2016.7417914> ISSN: 2376-8606.
- [60] Giosué Cataldo Marinó, Alessandro Petrin, Dario Malchiodi, and Marco Frasca. 2023. Deep neural networks compression: A comparative survey and choice recommendations. *Neurocomputing* 520 (Feb. 2023), 152–170. <https://doi.org/10.1016/j.neucom.2022.11.072>
- [61] Raul Mur-Artal and Juan D. Tardos. 2017. ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras. *IEEE Transactions on Robotics* 33, 5 (Oct. 2017), 1255–1262. <https://doi.org/10.1109/tro.2017.2705103> Publisher: Institute of Electrical and Electronics Engineers (IEEE).
- [62] Naveen Muralimanohar, Rajeev Balasubramanian, and Norman P Jouppi. 2009. CACTI 6.0: A tool to model large caches. *HP laboratories* 27 (2009), 28.
- [63] Kazuko Nishimura, Sanshiro Shishido, Yasuo Miyake, Masaaki Yanagida, Yoshiaki Satou, Makoto Shouho, Hidenari Kanehara, Ryota Sakaida, Yoshihiro Sato, Junji Hirase, Yuko Tomekawa, Yutaka Abe, Hiroshi Fujinaka, Yoshiyuki Matsunaga, Masashi Murakami, Mitsuru Harada, and Yasunori Inoue. 2018. An 8K4K-resolution 60fps 450ke-saturation-signal organic-photoconductive-film global-shutter CMOS image sensor with in-pixel noise canceller. In *2018 IEEE International Solid - State Circuits Conference - (ISSCC)*. 82–84. <https://doi.org/10.1109/ISSCC.2018.8310194> ISSN: 2376-8606.
- [64] NVIDIA. 2018. NVIDIA's xavier system-on-chip, *HotChips 30*. https://old.hotchips.org/hc30/1conf/1.12_Nvidia_XavierHotchips2018Final_814.pdf
- [65] Gabriel Nützi, Stephan Weiss, Davide Scaramuzza, and Roland Siegwart. 2011. Fusion of IMU and vision for absolute scale estimation in monocular SLAM. *Journal of intelligent & robotic systems* 61, 1-4 (2011), 287–299. Publisher: Springer.
- [66] onsemi. 2021. AP0100CS. <https://www.onsemi.com/products/sensors/image-signal-processors-isps/ap0100cs>
- [67] onsemi. 2021. AP0201AT. <https://www.onsemi.com/products/sensors/image-signal-processors-isps/ap0201at>
- [68] Angshuman Parashar, Minsoo Rhu, Anurag Mukkara, Antonio Puglielli, Rangharajan Venkatesan, Bruce Khailany, Joel Emer, Stephen W. Keckler, and William J. Dally. 2017. SCNN: An Accelerator for Compressed-sparse Convolutional Neural Networks. *ACM SIGARCH Computer Architecture News* 45, 2 (June 2017), 27–40. <https://doi.org/10.1145/3140659.3080254>
- [69] Andrzej Pietrasiewicz. 2020. Adding mainline Arm Frame Buffer Compression support for Rockchip. <https://www.collabora.com/news-and-blog/blog/2020/04/08/adding-mainline-arm-frame-buffer-compression-support-for-rockchip/>
- [70] Tong Qin, Peiliang Li, and Shaojie Shen. 2018. VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator. *IEEE Transactions on Robotics* 34, 4 (Aug. 2018), 1004–1020. <https://doi.org/10.1109/TRO.2018.2853729> Conference Name: IEEE Transactions on Robotics.
- [71] Mohammad Rastegari, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. 2016. XNOR-Net: ImageNet Classification Using Binary Convolutional Neural Networks. In *Computer Vision – ECCV 2016 (Lecture Notes in Computer Science)*, Bastian Leibe, Jiri Matas, Nicu Sebe, and Max Welling (Eds.). Springer International Publishing, Cham, 525–542. https://doi.org/10.1007/978-3-319-46493-0_32
- [72] Marc Riera, Jose-Maria Arnau, and Antonio Gonzalez. 2018. Computation Reuse in DNNs by Exploiting Input Similarity. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. 57–68. <https://doi.org/10.1109/ISCA.2018.00016> ISSN: 2575-713X.
- [73] Ethan Rublee, Vincent Rabaud, Kurt Konolige, and Gary Bradski. 2011. ORB: An efficient alternative to SIFT or SURF. In *2011 International conference on computer vision*. Ieee, 2564–2571.
- [74] Franyell Silfa, Gem Dot, Jose-Maria Arnau, and Antonio González. 2018. E-PUR: an energy-efficient processing unit for recurrent neural networks. In *Proceedings of the 27th International Conference on Parallel Architectures and Compilation Techniques (PACT '18)*. Association for Computing Machinery, New York, NY, USA, 1–12. <https://doi.org/10.1145/3243176.3243184>
- [75] Aaron Stillmaker and Bevan Baas. 2017. Scaling equations for the accurate prediction of CMOS device performance from 180 nm to 7 nm. *Integration* 58 (2017), 74–81. Publisher: Elsevier.
- [76] Synopsys. 2022. Synopsys Suite. <https://www.synopsys.com/>
- [77] Takafumi Taketomi, Hideaki Uchiyama, and Sei Ikeda. 2017. Visual SLAM algorithms: a survey from 2010 to 2016. *IPSN Transactions on Computer Vision and Applications* 9, 1 (2017), 16. Publisher: Springer.
- [78] Jie Tang, Bo Yu, Shaoshan Liu, Zhe Zhang, Weikang Fang, and Yanjun Zhang. 2018. /pi-SoC: Heterogeneous SoC Architecture for Visual Inertial SLAM Applications. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 8302–8307. <https://doi.org/10.1109/IROS.2018.8594181>
- [79] Raúl Taranco, José-María Arnau, and Antonio González. 2021. A low-power hardware accelerator for ORB feature extraction in self-driving cars. In *2021 IEEE 33rd International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*. IEEE, 11–21.
- [80] Raúl Taranco, José-María Arnau, and Antonio González. 2023. LOCATOR: Low-power ORB accelerator for autonomous cars. *J. Parallel and Distrib. Comput.* 174 (2023), 32–45. Publisher: Elsevier.
- [81] Nisarg Ujjainkar, Jingwen Leng, and Yuhao Zhu. 2023. ImaGen: A general framework for generating memory- and power-efficient image processing accelerators. In *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA '23)*. Association for Computing Machinery, Orlando, FL, USA. <https://arxiv.org/pdf/2304.03352>
- [82] Josh Weberuss, Lindsay Kleeman, David Boland, and Tom Drummond. 2017. FPGA acceleration of multilevel ORB feature extraction for computer vision. In *2017 27th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 1–8.
- [83] WikiChip. 2022. A14 Bionic - Apple. <https://en.wikichip.org/wiki/apple/ax/a14>
- [84] Junfei Xiao, Longlong Jing, Lin Zhang, Ju He, Qi She, Zongwei Zhou, Alan Yuille, and Yingwei Li. 2022. Learning From Temporal Gradient for Semi-Supervised Action Recognition. 3252–3262. https://openaccess.thecvf.com/content/CVPR2022/html/Xiao_Learning_From_Temporal_Gradient_for_Semi-Supervised_Action_Recognition_CVPR_2022_paper.html
- [85] Mengwei Xu, Mengze Zhu, Yunxin Liu, Felix Xiaozhu Lin, and Xuanzhe Liu. 2018. DeepCache: Principled Cache for Mobile Deep Vision. In *Proceedings of the 24th Annual International Conference on Mobile Computing and Networking (MobiCom '18)*. Association for Computing Machinery, New York, NY, USA, 129–144. <https://doi.org/10.1145/3241539.3241563>

- [86] Takayuki Yamashita and Kohji Mitani. 2013. 8K Extremely-High-Resolution Camera Systems. *Proc. IEEE* 101, 1 (Jan. 2013), 74–88. <https://doi.org/10.1109/JPROC.2012.2217371> Conference Name: Proceedings of the IEEE.
- [87] Ziyu Ying, Shulin Zhao, Haibo Zhang, Cyan Subhra Mishra, Sandeepa Bhuyan, Mahmut T. Kandemir, Anand Sivasubramaniam, and Chita R. Das. 2022. Exploiting Frame Similarity for Efficient Inference on Edge Devices. In *2022 IEEE 42nd International Conference on Distributed Computing Systems (ICDCS)*. 1073–1084. <https://doi.org/10.1109/ICDCS54860.2022.00107> ISSN: 2575-8411.
- [88] Khalid Yousif, Alireza Bab-Hadiashar, and Reza Hoseinnezhad. 2015. An Overview to Visual Odometry and Visual SLAM: Applications to Mobile Robotics. *Intelligent Industrial Systems* 1, 4 (Dec. 2015), 289–311. <https://doi.org/10.1007/s40903-015-0032-7>
- [89] Bo Yu, Wei Hu, Leimeng Xu, Jie Tang, Shaoshan Liu, and Yuhao Zhu. 2020. Building the Computing System for Autonomous Micromobility Vehicles: Design Constraints and Architectural Optimizations. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1067–1081. <https://doi.org/10.1109/MICRO50266.2020.00089>
- [90] Ekim Yurtsever, Jacob Lambert, Alexander Carballo, and Kazuya Takeda. 2020. A Survey of Autonomous Driving: Common Practices and Emerging Technologies. *IEEE Access* 8 (2020), 58443–58469. <https://doi.org/10.1109/ACCESS.2020.2983149> Conference Name: IEEE Access.
- [91] Zhe Zhang, Shaoshan Liu, Grace Tsai, Hongbing Hu, Chen-Chi Chu, and Feng Zheng. 2018. PIRVS: An Advanced Visual-Inertial SLAM System with Flexible Sensor Fusion and Hardware Co-Design. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*. 3826–3832. <https://doi.org/10.1109/ICRA.2018.8460672> ISSN: 2577-087X.
- [92] Yuhao Zhu, Anand Samajdar, Matthew Mattina, and Paul Whatmough. 2018. Euphrates: algorithm-SoC co-design for low-power mobile continuous vision. In *Proceedings of the 45th Annual International Symposium on Computer Architecture (ISCA '18)*. IEEE Press, Los Angeles, California, 547–560. <https://doi.org/10.1109/ISCA.2018.00052>